

第5章

监督学习算法

知识导图

视频



监督学习
算法



引言

在掌握经典智能算法与逻辑推理的问题求解逻辑后，本章将聚焦人工智能的核心学习范式——监督学习，这是实现机器从标注数据中学习规律、完成预测与分类任务的关键技术。监督学习以“数据标注 + 模型训练”为核心逻辑，通过学习输入特征与输出标签的映射关系，让模型具备对未知数据的预测能力，广泛应用于图像分类、语音识别、风险评估等实际场景。本章将系统拆解四类经典监督学习模型：K 最近邻凭借“近朱者赤”的实例推理逻辑，直观易懂且无需显式训练；决策树以树形结构模拟人类决策过程，兼具解释性与实用性；朴素贝叶斯基于贝叶斯定理与特征独立假设，实现高效的概率分类；支持向量机则通过最大化分类间隔，在高维数据中展现出强大的泛化能力。通过鸢尾花分类这一经典案例，你将亲手实现四类模型的完整流程，从数据预处理、模型构建到性能评估，对比分析不同模型在特征适应性、计算效率及泛化能力上的差异，掌握根据数据特性选择合适监督学习算法的核心逻辑，为后续处理更复杂的预测任务奠定坚实基础。

学习目标

(1) 理解监督学习的核心定义与“数据标注 + 模型训练”的核心逻辑，明确其在分类、回归等预测任务中的应用场景边界。

(2) 掌握 KNN、决策树、朴素贝叶斯、支持向量机的核心原理，包括距离度量、分裂准则、贝叶斯定理应用及核函数作用等关键机制。

(3) 熟练使用 Python 及 Scikit-learn 库实现四类模型，能够独立完成数据预处理、模型构建、超参数调优与结果验证的全流程操作。

(4) 学会从特征适应性、计算效率、泛化能力等维度对比四类模型的性能差异，能根据数据特性（如高维、线性/非线性）选择合适的监督学习算法。

(5) 具备运用监督学习解决实际分类问题的能力，通过鸢尾花分类等实战案例，掌握模型评估指标（准确率、混淆矩阵等）的解读与性能优化方法。

5.1 理论知识介绍

5.1.1 K 最近邻

1. 基本思想

K 最近邻（K-nearest neighbors, KNN）算法是一种基于距离的非参数监督学习方法，适用于分类和回归任务。对于给定的新数据点，KNN 通过查找训练集中与该点最相似的 K 个数据点来进行预测。KNN 不需要显式的训练阶段，因此是一种懒惰学习（lazy learning）算法。

2. 数学原理

距离度量：通常使用欧几里得距离（Euclidean distance）计算样本之间的相似性。对于两个特征向量 x_i 和 x_j ，其距离可以表示为

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2} \quad (5-1)$$

其他常用的距离度量包括曼哈顿距离、闵可夫斯基距离和余弦相似度。

分类规则：找到 K 个最近的邻居后，KNN通过简单的多数投票法决定新样本的类别。对于回归问题，则是通过取最近邻的均值来确定预测值。

3. 模型超参数

K 的选择：参数 K 对模型的性能有很大影响。较小的 K 值会导致模型对噪声敏感，容易过拟合，而较大的 K 值可能会使模型过于平滑，影响分类精度。 K 通常通过交叉验证来选定。

加权KNN：在一些情况下，KNN会对不同邻居赋予权重，更靠近的样本权重更大。这种方法可以通过距离的倒数来给出权重，公式为

$$w_i = \frac{1}{d(x, x_i)} \quad (5-2)$$

4. 优缺点

简单、无需假设数据分布，对小规模数据和噪声不多的数据表现较好。但随着样本量增加，预测时间显著增加，计算代价高；对特征尺度敏感，需进行标准化处理。

5.1.2 决策树

1. 基本思想

决策树（decision tree，DT）是一种树结构模型，通过递归分割数据来创建分支和叶子节点，最终形成一棵决策树，用于分类和回归任务。树的每一个内部节点表示对某个特征的判断，每个分支对应判断的结果，每个叶子节点表示类别或预测值。

2. 数学原理

决策树的核心在于选择最优的特征来分裂数据，常用的准则包括信息增益和基尼指数。

信息增益（ID3算法）：信息增益基于信息熵（entropy）的减少量。对于数据集 D ，其信息熵定义为

$$H(D) = - \sum_{i=1}^n p_i \log_2 p_i \quad (5-3)$$

基尼指数（CART算法）：基尼指数用来衡量数据集的纯度，基尼指数越小，数据集的纯度越高，决策树通过选择具有最小基尼指数的特征进行分裂。对于数据集 D ，其基尼指数定义为

$$\text{Gini}(D) = 1 - \sum_{i=1}^n p_i^2 \quad (5-4)$$

3. 剪枝

为了防止过拟合，决策树在生成后通常会进行剪枝，剪去一些对训练数据拟合良好但对测试数据无意义的节点。剪枝分为预剪枝和后剪枝：预剪枝即在生成过程中提前停止节点的生长，后剪枝即生成完整树后，通过验证集评估节点的重要性，去掉不必要的节点。

5.1.3 朴素贝叶斯

1. 基本思想

朴素贝叶斯（naive Bayes）是一种基于贝叶斯定理的分类算法。它假设特征之间相互独立，即特征的出现相互不影响，这一假设在实际应用中通常不成立，但朴素贝叶斯在文本分类等高维数据集上表现良好。

2. 数学原理

贝叶斯定理：朴素贝叶斯基于以下公式进行计算，其中 $P(C|X)$ 是给定特征 X 时类别 C 的后验概率， P

$P(X|C)$ 是类条件概率, $P(C)$ 是先验概率, $P(X)$ 是证据项。

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)} \quad (5-5)$$

条件独立性假设: 假设特征之间相互独立, 后验概率可以进一步表示为

$$P(C|X) \propto P(C) \prod_{i=1}^n P(x_i|C) \quad (5-6)$$

3. 优缺点

优点: 计算效率高, 适合高维度特征的数据; 对小规模的数据表现良好。

缺点: 由于条件独立假设的存在, 容易导致在特征之间相关性较强的情况下性能下降。

5.1.4 支持向量机

1. 基本思想

支持向量机 (support vector machine, SVM) 是一种用于分类和回归的强大算法。其目标是找到一个最优超平面, 将不同类别的样本尽可能地分开, 且保证分离的间隔最大, 从而提高泛化能力。

2. 数学原理

线性SVM: 假设训练数据线性可分, SVM的目标是找到一个超平面使得样本间隔最大化。对于线性可分的数据, 超平面方程为

$$w^T x + b = 0 \quad (5-7)$$

式中, w 为权重向量; b 为偏置。间隔最大化可以表示为一个约束优化问题, 目标是最小化:

$$\frac{1}{2} \|w\|^2 \quad (5-8)$$

约束条件为: $y_i(w^T x_i + b) \geq 1, i=1, 2, \dots, n$

软间隔与正则化: 对于实际数据不可完全分离的情况, SVM引入软间隔, 通过引入松弛变量 ζ_i 使得优化问题允许一定程度的错误分类, 同时加入正则化参数 C 来平衡间隔和错误分类的影响。

核函数: 当数据非线性可分时, SVM通过核函数将数据映射到更高维空间, 使其在高维空间中变得线性可分。常用核函数包括线性核、多项式核和径向基核 (RBF)。

3. 优缺点

优点: 在高维数据上效果良好, 适合复杂决策边界的分类任务; 通过选择合适的核函数可以处理非线性分类问题。

缺点: 训练时间较长, 对大规模数据不太适用; 对超参数 (如 C 和核函数参数) 敏感, 需通过交叉验证调整。

5.2 案例描述

5.2.1 案例背景介绍

本案例将使用鸢尾花 (Iris) 数据集进行分类任务。鸢尾花数据集是机器学习中最经典的数据集

之一，由英国统计学家和生物学家Fisher在1936年提出。数据集包含150条记录，分为三类，分别是Setosa、Versicolor和Virginica，每类各有50条记录。每条记录包含四个特征：花萼长度、花萼宽度、花瓣长度和花瓣宽度。

鸢尾花数据集因其简单、直观且适合可视化的特点，常用于分类算法的教学和实验。通过对鸢尾花数据集进行分类，我们可以学习如何使用不同的监督学习模型来解决分类问题，并比较这些模型的性能。

5.2.2 案例技术分析

本案例将采用逻辑回归、支持向量机、K最近邻和随机森林这四种模型对鸢尾花数据集进行分类。

K最近邻：通过计算样本之间的距离来进行分类，适合处理相对小规模的数据集，能够直观地展示基于实例的学习方式。

决策树：使用树形结构对数据进行决策分类，具有较强的解释性，适合处理类别型和连续型特征的数据。

朴素贝叶斯：基于贝叶斯定理和特征条件独立假设的分类方法，简单高效，适用于高维特征的数据，尤其是文本数据。

支持向量机：通过找到最优分类超平面来进行分类，擅长处理高维数据，并且能够通过核函数处理非线性分类任务。

5.3 设计与实现

本案例主要实现鸢尾花数据集分类。

Scikit-learn 是 Python 中最流行的机器学习库之一，提供了广泛的算法和工具，用于数据挖掘和数据分析。在开始之前，确保已经安装了 Scikit-learn。如果未安装，可以通过以下命令进行安装：

```
pip install scikit-learn
```

确保所有依赖的库也已安装，包括 NumPy 和 SciPy，这些库通常会与 Scikit-learn 一起自动安装。

5.3.1 数据集的预处理

在应用各种模型之前，我们首先需要对鸢尾花数据集进行预处理，预处理包括数据的加载、特征缩放和特征选择。注意，数据预处理是机器学习中的关键步骤，尤其是标准化（标准化到零均值和单位方差），这有助于加快模型收敛速度和提高准确性。

（1）加载数据集：加载用于模型学习的鸢尾花数据集。Scikit-learn库已经继承了这个数据集，直接导入即可：

```
from sklearn.datasets import load_iris
iris = load_iris()
X, y = iris.data, iris.target
```

（2）数据分割：为了评估模型的性能，需要将数据集分成训练集和测试集。通常，使用 70% 的数据作为训练集，剩余的 30% 作为测试集：

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
```

(3) 数据标准化：由于KNN算法依赖于距离的计算，不同的特征尺度可能会影响算法的性能。标准化将特征的均值调整为0，标准差调整为1，使各特征具有相同的尺度，有助于加快模型收敛并提高模型的准确性：

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5.3.2 使用 KNN 进行鸢尾花分类

手动编写实现KNN算法的部分核心功能，加深对模型的理解。

(1) 计算距离：编写一个函数来计算测试样本与所有训练样本之间的距离。这里通常使用欧几里得距离作为度量标准。

```
import numpy as np
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))
```

(2) 找到最近的 K 个邻居：对于每个测试样本，根据计算出的距离找出最近的 K 个邻居。这可以通过排序距离并选择前 K 个最小距离来实现。

```
def get_neighbors(X_train, test_sample, k):
    distances = [euclidean_distance(test_sample, x) for x in X_train]
    sorted_indices = np.argsort(distances)
    return sorted_indices[:k]
```

(3) 进行投票：对于分类问题，采用“多数投票”法来决定最终类别；对于回归问题，则采用平均法来预测结果。

```
def predict_classification(X_train, y_train, test_sample, k):
    neighbors_indices = get_neighbors(X_train, test_sample, k)
    neighbors_labels = [y_train[i] for i in neighbors_indices]
    prediction = max(set(neighbors_labels), key=neighbors_labels.count)
    return prediction
```

(4) 测试算法：分类准确率：使用测试集评估模型的性能。计算模型在测试集上的准确率，作为性能的评价指标。

```
y_pred = [predict_classification(X_train, y_train, test, 5) for test in X_test]
accuracy = np.sum(y_pred == y_test) / len(y_test)
print("Accuracy:", accuracy)
```

KNeighborsClassifier是Scikit-learn库提供的一个 K 最近邻算法实现，特别适合那些希望快速实现并测试 KNN 模型的开发者。在本案例中，我们将使用它与先前处理好的数据集展示在实际应用中快速部

署 KNN 模型的实现过程。

(5) 创建 KNN 分类器。

实例化 `KNeighborsClassifier`：接下来，创建 `KNeighborsClassifier` 类的一个实例。在这一步中，可以指定多个参数，如 `n_neighbors`（邻居数，即 K 的值）、`weights`（权重函数）和 `metric`（用于距离计算的度量标准）：

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3, weights='uniform', metric='euclidean')
```

(6) 训练模型：使用训练集数据来训练模型。

```
knn.fit(X_train_scaled, y_train)
```

(7) 模型评估。

使用测试集数据来评估模型的性能。`Scikit-learn` 提供了多种评估指标，如准确率：

```
from sklearn.metrics import accuracy_score
y_pred = knn.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

还可以使用其他更详细的性能评估方法，如混淆矩阵、精确度、召回率和 F1 分数：

```
from sklearn.metrics import classification_report, confusion_matrix
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

KNN 算法虽然简单，但通过适当的优化，可以显著提高其性能。下面是两个优化模型的例子：

交叉验证，这是评估模型泛化能力的一种技术，尤其是在参数调整过程中非常有用。它通过将数据分割成多个小子集，反复进行训练和验证，来模拟多种数据分布情况，从而提供关于模型性能的更稳健估计。

K-折交叉验证：数据集被分为 K 个子集，每个子集轮流作为测试集使用，而其他的 $K-1$ 个子集用作训练集，`Scikit-learn` 提供了简便的工具来实现这一方法。

(8) 调整 K 值：每次调整 K 值后进行交叉验证，从而找到最优的 K 值：

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(knn, X, y, cv=10, scoring='accuracy')
print(f"平均准确率: {scores.mean():.2f}")
```

(9) 距离权重：在计算投票时，可以给每个邻居赋予不同的权重，距离越近的样本权重越大。这样可以减少远距离样本对决策的影响，例如：

```
knn = KNeighborsClassifier(n_neighbors=k, weights='distance')
weights='distance' 表示按照距离的倒数进行加权，这样离新样本越近的邻居贡献更大
```

通过这个案例，读者不仅可以学习如何使用 `Scikit-learn` 来快速实现 KNN 模型，还能理解如何通过改变模型参数来优化性能和评估结果。这些技能在实际的机器学习项目中非常重要，能够帮助开发者高效地选择和调整模型以应对各种数据科学挑战。

5.3.3 使用决策树进行鸢尾花分类

在这个案例中，我们将尝试手工构建一个决策树，以便深入理解其构建过程及其在分类任务中的应用。手工构建决策树将帮助读者理解算法的每个细节，并为使用自动化工具提供坚实的基础。

1. 选择最佳分割特征

选择最佳分割特征是构建决策树的关键步骤。为此，我们需要评估每个特征对输出变量的预测能力。常用的方法包括信息增益和基尼指数。

信息增益：根据信息熵的减少来评估每个特征的效用。熵是度量数据集中随机变量不确定性的方式。特征选择的目标是找到那些使得分类后熵减少最多的特征。计算每个特征的信息增益，并选择信息增益最大的特征进行分割。

信息增益计算公式：

$$IG(D, f) = Entropy(D) - \sum_{v \in \text{Values}(f)} \frac{|D_v|}{|D|} Entropy(D_v) \quad (5-9)$$

式中， D 是数据集； f 是特征； D_v 是特征 f 取值为 v 的子集。

基尼指数：用于 CART（分类和回归树）算法，衡量数据集的不纯度。一个完全纯净的分类结果的基尼指数为 0。选择基尼指数最小的特征进行分割，从而达到快速清晰的分类。

基尼指数计算公式：

$$\text{Gini}(D) = 1 - \sum_{i=1}^n p_i^2 \quad (5-10)$$

式中， p_i 是选择类别 i 的概率。

我们将采用信息增益方法实现这一部分。

（1）定义计算熵的函数：决策树的构建过程依赖于信息增益，而信息增益的计算依赖于熵。

```
import numpy as np

def calculate_entropy(y):
    # 计算数据集中标签的熵
    class_counts = np.bincount(y)
    probabilities = class_counts / len(y)
    entropy = -np.sum([p * np.log2(p) for p in probabilities if p > 0])
    return entropy
```

（2）计算数据集分裂后的信息增益：信息增益是原始数据集的熵与划分后的加权熵之间的差值，用于选择最优特征进行划分。

```
def calculate_information_gain(y, y_left, y_right):
    # 计算信息增益
    p = len(y_left) / len(y)
    gain = calculate_entropy(y) - p * calculate_entropy(y_left) - (1 - p) * calculate_entropy(y_right)
    return gain
```


2. 寻找最优的划分点

遍历每个特征，并针对特征的每一个可能的划分点计算信息增益，找到使信息增益最大的特征和划分点作为最优划分。

```
def find_best_split(X, y):
    best_gain = 0
    best_split = None
    n_features = X.shape[1]

    for feature_index in range(n_features):
        values = X[:, feature_index]
        unique_values = np.unique(values)

        for threshold in unique_values:
            left_indices = values <= threshold
            right_indices = values > threshold

            y_left = y[left_indices]
            y_right = y[right_indices]

            if len(y_left) == 0 or len(y_right) == 0:
                continue

            gain = calculate_information_gain(y, y_left, y_right)

            if gain > best_gain:
                best_gain = gain
                best_split = {
                    'feature_index': feature_index,
                    'threshold': threshold,
                    'left_indices': left_indices,
                    'right_indices': right_indices
                }

    return best_gain, best_split
```

3. 构建递归树节点

递归过程是决策树构建中的核心。每一个子集将使用相同的过程继续进行分割，直到满足某个停止条件：

- (1) **深度限制**：为防止树变得过深而导致过拟合，可以设置树的最大深度。
- (2) **样本属于同一类别**：如果当前节点的数据集中的所有样本都属于同一个类别，那么该节点即

为叶子节点，无须再继续分裂。

(3) 信息增益为零：如果划分后的信息增益为零，表示当前节点的数据无法进一步划分或继续划分无法增加信息增益，这时也停止分裂，将当前节点作为叶子节点。

这个递归过程将持续进行，直到所有有效的分割完成或达到预设的停止条件。

```
class DecisionTreeNode:
    def __init__(self, feature_index=None, threshold=None, left=None, right=None,
value=None):
        self.feature_index = feature_index
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

def build_tree(X, y, depth=0, max_depth=3):
    n_samples, n_features = X.shape
    n_labels = len(np.unique(y))

    # 停止条件
    if n_labels == 1 or depth >= max_depth:
        leaf_value = np.bincount(y).argmax()
        return DecisionTreeNode(value=leaf_value)

    # 找到最佳分裂
    gain, split = find_best_split(X, y)

    if gain == 0:
        leaf_value = np.bincount(y).argmax()
        return DecisionTreeNode(value=leaf_value)

    left_subtree = build_tree(X[split['left_indices']], y[split['left_indices']],
depth + 1, max_depth)
    right_subtree = build_tree(X[split['right_indices']], y[split['right_indices']],
depth + 1, max_depth)

    return DecisionTreeNode(feature_index=split['feature_index'], threshold=split
['threshold'], left=left_subtree, right=right_subtree)
```

4. 通过决策树来进行预测

(1) 定义预测函数：

```
def predict(sample, tree):
    if tree.value is not None:
```

```

        return tree.value

    feature_value = sample[tree.feature_index]
    if feature_value <= tree.threshold:
        return predict(sample, tree.left)
    else:
        return predict(sample, tree.right)

```

(2) 在数据集上测试决策树:

```

tree = build_tree(X_train, y_train, max_depth=3)

# 测试决策树
predictions = [predict(sample, tree) for sample in X_test]

# 计算准确率
accuracy = np.mean(predictions == y_test)
print(f" 决策树分类准确率: {accuracy:.2f}")

```

5. 使用Sckit-learn构建决策树分类器

接下来使用 Sckit-learn 提供的 DecisionTreeClassifier 类来构建、训练和评估一个决策树分类器。

(1) **创建决策树实例**: 在这一步, 可以设置多个参数来定义树的结构, 例如 max_depth (树的最大深度) 和 min_samples_split (分割内部节点所需的最小样本数)。

```

from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(max_depth=5, min_samples_split=4)

```

(2) **训练模型**。使用训练集数据训练决策树模型。

```

clf.fit(X_train, y_train)

```

(3) **模型评估**。使用测试集来评估模型的性能。Scikit-learn提供了多种评估指标, 其中准确率是最常见的一种。

```

from sklearn.metrics import accuracy_score

y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of the model: {accuracy:.2f}')

```

此外, 可以使用混淆矩阵、召回率和 F1 分数等其他指标来更全面地评估模型性能。

```

from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

6. 优化决策树模型

在机器学习模型的开发过程中, 优化是至关重要的步骤, 特别是对于决策树模型而言, 恰当的优

化不仅可以提高模型的准确性，还能防止过拟合，从而提高模型在未见数据上的泛化能力。在本案例中，我们将探讨两种主要的决策树优化技术：树的剪枝和交叉验证。

(1) 树的剪枝：剪枝是减少决策树复杂度、防止过拟合的有效手段。在 Scikit-learn 中，决策树提供了两种剪枝参数：

① 预剪枝：预剪枝通过提前停止树的构造而减少模型的复杂性。常见的预剪枝参数包括：

- `max_depth`：限制树的最大深度。
- `min_samples_split`：一个节点必须具有至少`min_samples_split`个样本才能被允许分裂。
- `min_samples_leaf`：叶节点必须有`min_samples_leaf`个样本。

这些参数可以直接在创建`DecisionTreeClassifier`时设置：

```
from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier(max_depth=5, min_samples_split=4, min_samples_leaf=2)
clf.fit(X_train, y_train)
```

② 后剪枝：后剪枝在树完全生成后进行，通过评估是否去除某些分支能改善测试集性能来执行。在 Scikit-learn 中，可以使用 `ccp_alpha` 参数控制后剪枝的复杂性成本参数。

```
clf = DecisionTreeClassifier(ccp_alpha=0.01)
clf.fit(X_train, y_train)
```

增加`ccp_alpha`的值会增加剪枝的数量，通常需要通过交叉验证来找到最优的`ccp_alpha`值。

(2) 交叉验证：

```
from sklearn.model_selection import cross_val_score
clf = DecisionTreeClassifier(max_depth=5)
scores = cross_val_score(clf, X, y, cv=5) # 5-fold cross-validation
print("Accuracy: %0.2f (+/- %0.2f)" % (scores.mean(), scores.std() * 2))
```

通过上述方法，可以系统地调整和评估决策树模型，找到最佳的模型参数和结构。这不仅增强了模型对新数据的预测能力，也使模型更加健壮，避免了因训练数据的偶然性质而导致的过拟合问题。

5.3.4 使用朴素贝叶斯进行鸢尾花分类

1. 概率计算

朴素贝叶斯算法的概率计算涉及计算先验概率和条件概率。具体地，对于一个新的样本，需要计算其在每个类别下的后验概率，然后根据后验概率的大小来确定其所属的类别。

(1) 先验概率：给定类别的先验概率可以通过训练集中相应类别的样本数量除以总样本数量来计算。

$$P(C_i) = \frac{\text{类别 } C_i \text{ 的样本数量}}{\text{总样本数量}} \quad (5-11)$$

```
def calculate_prior(y):
    classes = np.unique(y)
    prior_probabilities = {}
    for cls in classes:
```

```
prior_probabilities[cls] = np.sum(y == cls) / len(y)
return prior_probabilities
```

```
priors = calculate_prior(y_train)
print(" 先验概率 :", priors)
```

(2) **条件概率**：在朴素贝叶斯算法中，假设每个特征在给定类别的条件下是相互独立的。因此，可以将条件概率计算为每个特征在给定类别下出现的概率的乘积。

$$P(X_j|C_i) = \frac{\text{类别 } C_i \text{ 中特征 } X_j \text{ 的样本数量}}{\text{类别 } C_i \text{ 的样本总数}} \quad (5-12)$$

对于连续特征，朴素贝叶斯假设特征符合正态分布，因此可根据各类别特征的均值和方差计算对应的条件概率。

```
def calculate_mean_variance(X, y):
    classes = np.unique(y)
    mean_variance = {}
    for cls in classes:
        X_class = X[y == cls]
        mean_variance[cls] = {
            "mean": np.mean(X_class, axis=0),
            "var": np.var(X_class, axis=0)
        }
    return mean_variance

mean_variance_params = calculate_mean_variance(X_train_scaled, y_train)
print(" 均值和方差 :", mean_variance_params)
```

计算概率密度函数：

```
def gaussian_probability(x, mean, var):
    eps = 1e-6 # 防止方差为零的数值问题
    coefficient = 1.0 / np.sqrt(2.0 * np.pi * var + eps)
    exponent = np.exp(-((x - mean) ** 2) / (2.0 * var + eps))
    return coefficient * exponent
```

(3) **后验概率**：根据贝叶斯定理，可以计算文本样本属于每个类别的后验概率。

$$P(C_i|X) = \frac{P(X|C_i) \cdot P(C_i)}{P(X)} \quad (5-13)$$

式中， $P(C_i|X)$ 表示在给定样本 X 的情况下，属于类别 C_i 的后验概率； $P(X|C_i)$ 是在类别 C_i 下观察到样本 X 的条件概率； $P(C_i)$ 是类别 C_i 的先验概率； $P(X)$ 是样本 X 的边缘概率。

对于每个测试样本，计算它属于每个类别的概率，并选择概率最大的类别作为预测结果：

```
def predict(X, priors, mean_variance_params):
    predictions = []
```

```
classes = list(priors.keys())
for i in range(X.shape[0]):
    posteriors = {}
    for cls in classes:
        prior = np.log(priors[cls]) # 取对数防止浮点数下溢
        conditional = 0
        for j in range(X.shape[1]):
            mean = mean_variance_params[cls]["mean"][j]
            var = mean_variance_params[cls]["var"][j]
            conditional += np.log(gaussian_probability(X[i, j], mean, var))
        posteriors[cls] = prior + conditional
    predictions.append(max(posteriors, key=posteriors.get))
return np.array(predictions)

y_pred = predict(X_test_scaled, priors, mean_variance_params)
```

朴素贝叶斯是一种简单而有效的分类算法，在文本分类等任务中得到了广泛的应用。Scikit-learn 也提供了 MultinomialNB 类来实现朴素贝叶斯分类器，尝试使用它来快速实现朴素贝叶斯模型：

```
from sklearn.naive_bayes import GaussianNB
# 朴素贝叶斯模型训练
nb_model = GaussianNB()
nb_model.fit(X_train_scaled, y_train)
# 预测
y_pred = nb_model.predict(X_test_scaled)
```

分别使用准确率、混淆矩阵、召回率和F1分数等指标评估模型。

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report, confusion_matrix

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of the model: {accuracy:.2f}')
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

2. 朴素贝叶斯性能优化

朴素贝叶斯算法在实践中通常需要进行性能优化，以提高其分类效果。在本案例中，将介绍介绍几种常用的性能优化方法：

1) 特征优化

过滤无用或冗余的特征可以提高朴素贝叶斯的性能，因为该算法假设特征之间相互独立。使用特征选择算法，如 SelectKBest 或基于信息增益的特征筛选方法，可以减少模型的复杂度并提高泛化能力。对于特征较多的高维数据，可以使用主成分分析（PCA）等降维方法减少特征数量，从而减少噪声，提高模型的鲁棒性和效率。

2) 参数调整

朴素贝叶斯算法有一些重要的参数需要调整, 其中最重要的是平滑参数。平滑参数用于解决概率估计中的零概率问题, 避免出现概率为零的情况。常用的平滑参数包括拉普拉斯平滑和 Lidstone 平滑等。通过调整平滑参数, 可以改善模型在训练数据上的拟合效果, 从而提高其在测试数据上的泛化能力。

通过对特征优化和参数调整, 可以进一步优化朴素贝叶斯算法的性能, 提高其在实际应用中的分类效果。这将有助于我们更好地应用朴素贝叶斯算法解决实际的文本分类等问题, 并为后续模型改进提供参考。

3. 模型改进

朴素贝叶斯有多个变体, 每种适合不同的数据特性, 可以根据数据类型选择合适的朴素贝叶斯分类器:

高斯朴素贝叶斯 (GaussianNB): 适合特征是连续变量且符合正态分布的情况。

多项式朴素贝叶斯 (MultinomialNB): 适合特征是离散变量的情况, 尤其是文本分类。

伯努利朴素贝叶斯 (BernoulliNB): 适合特征是二元变量的情况, 如词袋模型。

5.3.5 使用支持向量机进行鸢尾花分类

在这个案例中, 我们将实现一个简单的支持向量机算法, 以完成一个二分类任务。以下是详细的步骤:

1. 加载并准备数据

为了方便可视化和简化实现, 我们在鸢尾花数据集中选择两个类别和两个特征, 转为解决一个二分类问题。

SVM通常处理二分类任务, 将标签转换为 -1 和 1。

```
# 加载数据集
iris = load_iris()
X = iris.data[:100, :2] # 选择前 100 条数据和前两个特征
y = iris.target[:100]   # 仅选择两类鸢尾花 (Setosa 和 Versicolor)

# 标签转换: 将标签 0 保留为 -1, 将标签 1 转换为 1
y = np.where(y == 0, -1, 1)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=34)
```

定义两个核函数: 线性核函数和RBF核函数。

```
# 线性核函数
def linear_kernel(x1, x2):
    return np.dot(x1, x2)

# RBF 核函数
```

```
def rbf_kernel(x1, x2, gamma=0.5):  
    return np.exp(-gamma * np.linalg.norm(x1 - x2) ** 2)
```

2. 实现支持向量机算法

为了实现支持向量机算法，需要定义一个支持向量机的类，其中包括核函数的选择、参数的初始化、损失函数的定义等功能。

```
class SVM:  
    def __init__(self, kernel=linear_kernel, learning_rate=0.001, lambda_param=0.01,  
n_iters=100):  
        self.kernel = kernel  
        self.learning_rate = learning_rate  
        self.lambda_param = lambda_param  
        self.n_iters = n_iters  
        self.alpha = None  
        self.b = 0  
        self.X = None  
        self.y = None  
  
    def fit(self, X, y):  
        n_samples, n_features = X.shape  
        self.alpha = np.zeros(n_samples)  
        self.X = X  
        self.y = y  
  
        # 梯度下降优化  
        for _ in range(self.n_iters):  
            for i in range(n_samples):  
                condition = y[i] * (self._decision_function(X[i])) < 1  
                if condition:  
                    self.alpha[i] += self.learning_rate * (1 - y[i] * self._decision_  
function(X[i]))  
  
            # 更新偏置 b  
            support_vectors_idx = np.where(self.alpha > 1e-5)[0]  
            self.b = np.mean([y[i] - self._decision_function(X[i]) for i in support_vectors_  
idx])  
  
    def _decision_function(self, x):  
        return np.sum([self.alpha[i] * self.y[i] * self.kernel(self.X[i], x) for i in  
range(len(self.alpha))]) - self.b
```

```
def predict(self, X):
    return np.sign([self._decision_function(x) for x in X])
```

3. 模型训练和预测

在训练模型的步骤中，使用梯度下降法或其他优化算法来更新模型参数，使得损失函数最小化。使用线性核和RBF核分别训练模型，并评估测试集上的准确率：

```
# 使用线性核初始化并训练模型
svm_linear = SVM(kernel=linear_kernel, learning_rate=0.001, lambda_param=0.01, n_iters=100)
svm_linear.fit(X_train, y_train)

# 使用 RBF 核初始化并训练模型
svm_rbf = SVM(kernel=rbf_kernel, learning_rate=0.001, lambda_param=0.01, n_iters=100)
svm_rbf.fit(X_train, y_train)

# 预测并评估模型（线性核）
predictions_linear = svm_linear.predict(X_test)
accuracy_linear = np.mean(predictions_linear == y_test)
print(f"线性核模型的测试集准确率：{accuracy_linear:.2f}")

# 预测并评估模型（RBF 核）
predictions_rbf = svm_rbf.predict(X_test)
accuracy_rbf = np.mean(predictions_rbf == y_test)
print(f"RBF 核模型的测试集准确率：{accuracy_rbf:.2f}")
```

4. 可视化决策边界

为了展示两种核函数对决策边界的影响，我们分别绘制线性核和RBF核的决策边界。

```
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
def plot_decision_boundary(X, y, model, title):
    x0, x1 = np.meshgrid(
        np.linspace(X[:, 0].min() - 1, X[:, 0].max() + 1, 200),
        np.linspace(X[:, 1].min() - 1, X[:, 1].max() + 1, 200)
    )
    X_grid = np.array([x0.ravel(), x1.ravel()]).T
    y_pred = model.predict(X_grid).reshape(x0.shape)

    cmap_background = ListedColormap(['#FFAAAA', '#AAAAFF'])
    cmap_points = ListedColormap(['#FF0000', '#0000FF'])

    plt.contourf(x0, x1, y_pred, alpha=0.3, cmap=cmap_background)
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap_points, edgecolor='k', marker='o',
                s=50)
```

```
plt.xlabel('特征1')
plt.ylabel('特征2')
plt.title(title)
plt.show()

# 绘制线性核的决策边界
plot_decision_boundary(X, y, svm_linear, 'SVM 决策边界 (线性核)')

# 绘制 RBF 核的决策边界
plot_decision_boundary(X, y, svm_rbf, 'SVM 决策边界 (RBF 核)')
```

图5-1和图5-2是程序绘制的分别采用线性核函数和RBF核函数时，分类任务的决策边界。

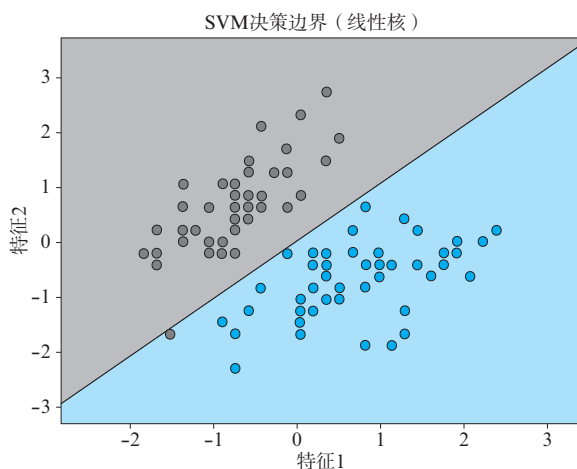


图 5-1 SVM 决策边界 (线性核)

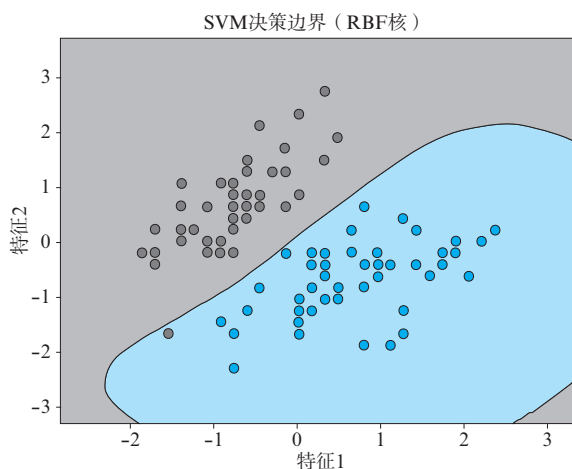


图 5-2 SVM 决策边界 (RBF 核)

5. 使用Scikit-learn快速实现SVM分类器

下面使用sklearn提供的SVC来快速实现支持向量机分类器：

(1) **创建支持向量机模型**：使用Scikit-learn中的SVC类创建支持向量机分类器，设置适当的参数，如核函数、正则化参数等。

```
from sklearn.svm import SVC
# 创建支持向量机模型
svm_model = SVC(kernel='linear', C=1.0, random_state=34)
```

这里使用线性核函数，并设置了正则化参数C为1.0。

(2) **训练模型**：使用训练数据对支持向量机模型进行训练。

```
# 训练模型
svm_model.fit(X_train, y_train)
```

(3) **模型评估**：使用测试数据评估模型的性能，计算分类准确率、精确度、召回率等指标。

```
# 模型评估
y_pred = svm_model.predict(X_test)
```

```

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)

```

通过这些步骤，可以快速创建、训练和评估支持向量机模型，从而得到模型在测试数据上的性能指标。

6. SVM的多分类实现

在前述案例中，为简化演示过程，将鸢尾花分类任务限定为二分类场景。本节将进一步介绍支持向量机扩展至多分类任务的实现方式。

SVM最初是为二分类问题设计的，但通过一些策略，SVM 可用于多分类问题。Scikit-learn 中的 SVM 已经集成了多分类策略，因此可以直接使用 SVC 进行多分类。在实现原理上，SVM 使用了两种常见的方法来扩展到多分类：一对一（One-vs-One, OvO）和一对多（One-vs-Rest, OvR）：

1) 一对多

概念：在 OvR 中，将一个类与其他所有类进行区分，每次训练一个二分类器，最终一共训练 N 个分类器（ N 是类的数量）。在预测时，将输入数据应用到每一个分类器上，最后选择置信度最高的那个分类器对应的类作为最终预测结果。

优点：实现相对简单，计算量较小。

缺点：可能存在类之间置信度相似的情况，导致分类结果不够准确。

2) 一对一

概念：在 OvO 中，针对每两个类构建一个二分类器。如果数据集有 N 个类，那么需要构建 $\frac{N(N-1)}{2}$ 个二分类器。预测时，将输入数据应用到所有分类器上，通过投票机制选择得票最多的类别。

优点：通常性能更好，因为分类器仅需区分两类，所以决策边界更加精确。

缺点：需要训练大量分类器，计算量较大。

有兴趣的读者可以自行查阅sklearn的源代码，研究其中SVC对于多分类的具体实现。

7. SVM的性能优化

下面介绍几种常用的SVM优化手段：

1) 核函数的选择

SVM 使用核函数将数据映射到高维空间，以找到一个可以线性分离的数据表示。不同的核函数适用于不同的数据集，因此选择合适的核函数对于 SVM 的性能至关重要。

（1）线性核（linear kernel）适用于线性可分的数据，或者在高维空间中容易被线性分离的数据；优点是训练速度快，计算量小，如果数据集特征数远大于样本数（例如文本数据），线性核可能是更合适的选择。

（2）多项式核（polynomial kernel）用于处理多次方关系的数据，非线性程度通过核参数 degree 来控制，如果存在明确的多项式关系，这种核函数可能表现良好。

（3）RBF核（径向基函数核）是最常用的核函数之一，适用于大多数非线性数据，可以将数据映射

到无限维空间，通过 γ 控制高斯分布的“宽度”，使得模型能够处理更加复杂的边界。

(4) Sigmoid核类似于神经网络的激活函数，在某些情况下，可以将 SVM 转化为神经网络。

2) 参数调优

SVM 的性能与模型的超参数密切相关，尤其是正则化参数 C 和核参数 γ （对于 RBF 核）。

(1) 正则化参数 C 是控制训练错误和模型复杂度的权衡参数。较大的 C 值对误分类有严格的惩罚，意味着模型会更注重准确分类训练集，可能会过拟合；较小的 C 值对误分类的容忍度更高，使模型更具泛化能力，可能会欠拟合。

(2) 核参数 γ （RBF核）决定单个训练样本对决策边界的影响范围。较大的 γ 导致每个样本的影响范围很小，模型复杂度高，容易过拟合；较小的 γ 导致影响范围大，模型较为简单，可能会欠拟合。

3) 网格搜索和随机搜索

用于调整机器学习模型超参数的系统化方法，通过对模型的多个参数组合进行尝试，找到使模型在给定数据集上表现最佳的超参数组合。网格搜索会尝试各种可能的参数组合，随机搜索不是尝试所有参数组合，而是在给定范围内随机选取一定数量的组合进行尝试。下面是使用网格搜索和随机搜索对 SVM 进行调优的实例：

(1) 网格搜索：

```
from sklearn.model_selection import GridSearchCV

# 定义参数网格
param_grid = {
    'kernel': ['linear', 'poly', 'rbf', 'sigmoid'],
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001]
}

# 创建支持向量机分类器
svm_model = SVC(random_state=34)

# 使用网格搜索交叉验证找到最优参数组合
grid_search = GridSearchCV(estimator=svm_model, param_grid=param_grid,
                           cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# 输出最优参数
print("Best parameters:", grid_search.best_params_)
```

(2) 随机搜索：

```
from sklearn.model_selection import RandomizedSearchCV

# 使用 RandomizedSearchCV 进行随机参数调优
```

```
random_search = RandomizedSearchCV(SVC(), param_grid, n_iter=10, verbose=2, cv=5,
random_state=34)
random_search.fit(X_train, y_train)

# 输出最佳参数
print("Best parameters:", random_search.best_params_)
```

通过这些方法，我们可以对支持向量机模型进行较完善的优化，在提高模型性能的同时也能够深入了解支持向量机算法在不同情况下的表现。

总 结

本章深入探讨了几种经典的机器学习算法，包括K近邻、决策树、朴素贝叶斯和支持向量机。通过对这些算法的学习，我们可以获得以下几点结论：

首先，KNN算法是一种直观且易于理解的机器学习算法，适用于分类和回归问题。通过计算样本之间的距离来进行预测，KNN算法的性能在很大程度上取决于K值的选择和距离度量方式。

其次，决策树算法通过模拟人类的决策过程来进行预测，具有易于解释和理解的特点。通过构建树形结构来表示决策过程，决策树算法在处理分类和回归问题时表现出良好的性能。

再次，朴素贝叶斯算法基于贝叶斯定理，通过假设特征之间相互独立来简化计算，适用于处理大规模数据集和文本分类等任务。尽管其对特征独立性的假设可能不太符合实际情况，但朴素贝叶斯算法仍然在许多领域中得到广泛应用。

最后，支持向量机是一种强大的机器学习算法，通过在高维空间中找到一个最优的超平面来实现分类或回归。支持向量机具有很强的泛化能力和分类性能，在处理复杂问题时表现出色。

通过本章的学习，我们不仅深入理解了这些经典算法的原理和实现方式，还学会了如何使用Python中的机器学习库来快速实现和调优这些算法。这些知识和技能将为我们在实际问题中应用机器学习算法提供坚实的基础。

习 题

1. 试比较K最近邻算法与支持向量机算法在处理高维数据时的优劣势。请结合算法基本原理，分析两种算法的计算复杂度、对数据规模的敏感性，并论述其适用的应用场景。

2. 选取一个非鸢尾花数据集的实际问题，应用本章所学的四种监督学习算法（K最近邻、决策树、朴素贝叶斯、支持向量机）实现分类任务。对各算法在该数据集上的性能进行定量评估，通过特征选择、参数优化等方法提升模型性能。分析影响各算法效能的关键因素，并探讨在实际应用场景中如何进行算法选择与优化。

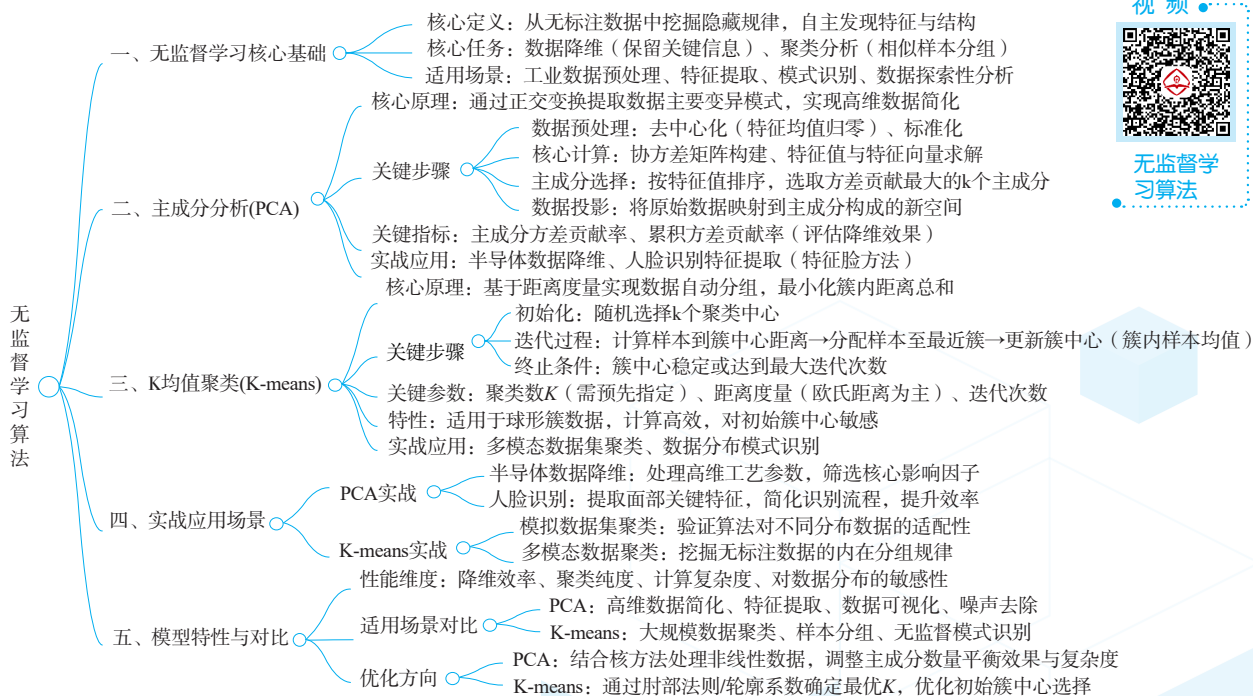
扩展思考

1. 了解常见的集成学习方法，如随机森林（random forest）、梯度提升树（gradient boosting）、极端梯度提升（XGBoost）等，并了解它们如何通过多个弱分类器提高模型的准确性。
2. 了解和实践特征选择（如递归特征消除 RFE、LASSO）和降维方法（如 PCA、LDA）。理解它们在减小模型复杂度、降低过拟合风险方面的作用。
3. 了解深度学习框架中的经典监督学习模型（如卷积神经网络、循环神经网络），并探索它们在图像分类和序列预测中的应用。

第6章

无监督学习算法

知识导图



视频



无监督学习算法

引言

在掌握监督学习的“数据标注 + 模型训练”逻辑后，本章将聚焦人工智能的另一核心学习范式——无监督学习，这是从无标注数据中挖掘隐藏规律、实现特征自主发现的关键技术。无监督学习无需人工标注标签，通过自动探索数据的内在结构与分布特征，完成降维、聚类等核心任务，广泛应用于数据预处理、模式识别、特征提取等实际场景。本章将系统拆解两类经典无监督学习算法：主成分分析通过正交变换提取数据的主要变异模式，在保留核心信息的同时实现高维数据简化；K均值聚类基于距离度量实现数据的自动分组，让相似样本自然聚集形成簇群。通过半导体数据降维、人脸识别特征提取、多模

态数据聚类三大案例任务，你将亲手实现两类算法的完整流程，从数据预处理、模型构建到结果可视化，对比分析它们在特征提取、数据分组上的核心逻辑，掌握根据数据特性选择无监督学习策略的能力，为后续处理高维无标注数据、优化模型输入奠定坚实基础。

学习目标

(1) 理解无监督学习的核心定义与“自主挖掘数据规律”的核心逻辑，明确其在数据降维、聚类分析等任务中的应用场景边界。

(2) 掌握主成分分析与 K 均值聚类的核心原理，包括主成分分析的方差最大化准则、特征值与特征向量求解，以及 K 均值聚类的簇内距离最小化、聚类中心迭代更新机制。

(3) 熟练使用 Python 及 NumPy、Scikit-learn 库实现两类算法，能够独立完成数据预处理、模型构建、结果可视化与性能评估的全流程操作。

(4) 学会从降维效率、聚类纯度、计算复杂度等维度对比两类算法的性能差异，能根据数据特性（如高维、线性 / 非线性、分布模式）选择合适的无监督学习策略。

(5) 具备运用无监督学习解决实际问题的能力，通过半导体数据降维、人脸识别特征提取等实战案例，掌握主成分贡献度解读、聚类效果评估及算法参数调优技巧。

6.1 理论知识介绍

6.1.1 PCA

1. PCA案例背景

在半导体制造过程中，设备传感器会采集数百个维度的测量参数，这些高维数据不仅包含关键工艺特征，也掺杂了大量冗余信息和噪声。直接分析此类数据面临维度灾难和计算效率低下的问题，同时可能掩盖核心变量的作用。主成分分析作为一种经典的无监督降维方法，能够通过正交变换提取数据的主要变化模式，帮助工程师聚焦关键工艺参数，优化生产流程。例如，在半导体良率分析中，PCA可识别影响产品性能的核心变量，为工艺改进提供数据支持。

2. PCA基本原理

主成分分析（principal component analysis, PCA）是一种经典的无监督机器学习算法。其核心目标是通过正交线性变换，将原始高维空间中的相关变量转化为低维空间中的一组互不相关的主成分，从而实现数据降维。具体而言，PCA 从 n 维原始特征中提取 k 个（ $k < n$ ）具有最大方差贡献的正交方向作为新坐标轴，通过投影操作保留数据的主要变异模式，同时剔除冗余特征与噪声干扰。这种特性使其能够从高维数据的复杂结构中抽提本质特征，揭示数据内在的简洁分布规律。

有一组100个二维样本数据，它的分布如图6-1所示。

通过观察二维样本的分布特性，可量化分析各维度的信息贡献差异。数据显示， X 轴方向的数据变化范围显著（0~6），而 Y 轴方向的变异程度极低（0.5~0.8）。在统计学视角下，方差是衡量数据分布离散程度的核心指标，直接反映特征的信息熵。 Y 轴的低方差表明其维度内数据趋同性高，所承载的差异性信息可归类为次要成分或噪声。

基于此，主成分分析通过正交变换将数据投影至方差最大的方向（即 X 轴方向），从而保留数据集中95%以上的信息量（假设示例值）。通过剔除 Y 轴的冗余维度，算法将原始二维数据压缩至一维特征空间。这一降维过程不仅降低了数据维度复杂性，还通过去除噪声提升了后续机器学习模型的鲁棒性。

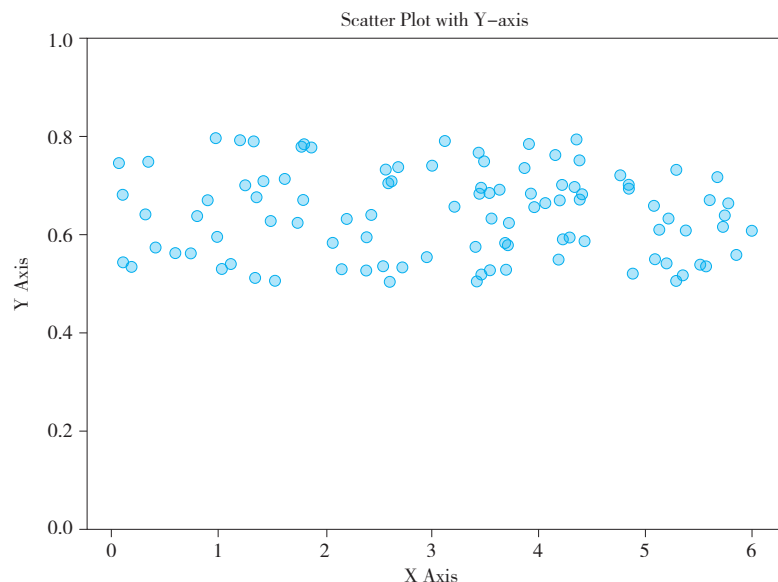


图 6-1 数据分布图

再比如图6-2中的数据分布：

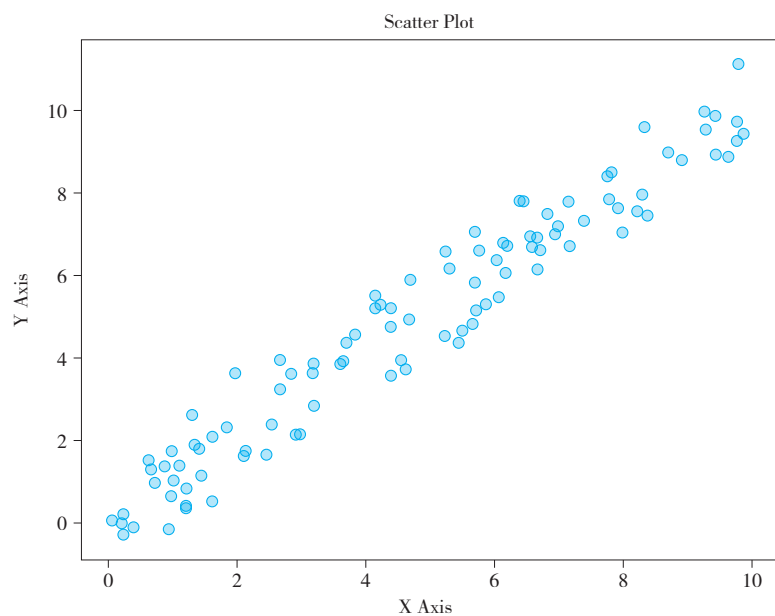


图 6-2 数据分布图

其样本数据分布在 X 轴上的变化程度和在 Y 轴上的变化程度无法明显区分，从而无法判断出哪个信息对特征影响大，此时可以尝试转动坐标轴进行降维，如图6-3所示。就可以明显看出此例和上面那个例子一样了。

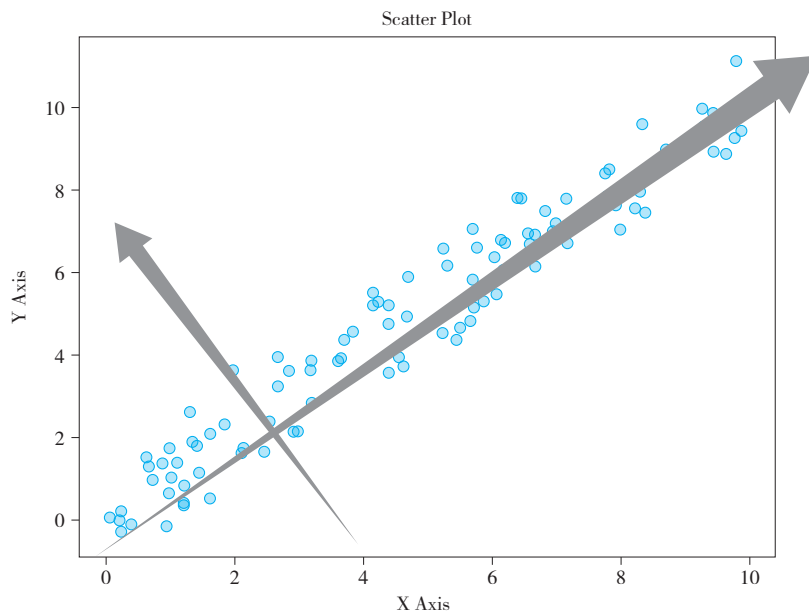


图 6-3 新建坐标图

由此可以看出，样本本身的具体数据并不是关键因素，关键因素是样本数据之间的关系和数据的整体分布。原来的数据是在直角坐标系下，当换一个坐标系来表示时，本质上相当于对数据进行了一次正交变换（从数学公式看），在新的坐标系下，我们能更清楚地看到数据的特点，这为后续进一步操作数据提供了可能。PCA求出了一个正交矩阵 P ，然后用这个矩阵对数据进行正交变换得到新的数据。

$$Y = PX \quad (6-1)$$

正交变换就相当于换了一个坐标系，所以其结果就是我们换了一个坐标系来观察数据。假如在这个坐标系下取前 k 个变化最大的轴上的数据，这就实现了降维。按照两阶段来理解PCA会容易得多，简单来说就是：第一阶段找了一个新的坐标系来表示数据，这个新的坐标系要求能最大限度地展示每个轴上的数据变化大小；第二阶段在新坐标系下取前 k 个变化最大的轴上的数据，从而实现降维。

那么怎么才能找到这个新的坐标系呢？这就需要回到PCA所要解决的问题上。

PCA最本质的功能就是提取数据的主要信息，高维的数据存在维度灾难，而且许多变量之间可能存在相关性，也可能存在噪声，通过提取其中的主要信息来实现降维就是一种有效的解决手段。这里的“主要”指的是数据中包含的信息量较大的部分，那么一个很自然的问题就是：怎么来度量信息量的大小？

上面两个例子，我们是用数据在某个轴上的变化大小，也就是分散程度来度量信息量的大小，这显然可以用数学上的方差来进行量化。

有了度量指标，接下来就是找到新的坐标系，就要逐个找到每个坐标轴，所以目标就是先在整个数据空间找到一个坐标轴（方向），使得数据在这个坐标轴上的投影（投影=坐标值）的方差达到最大，这个方向就是新的坐标系的第一个轴；然后再找一个方差次大的，而且与第一个轴垂直的方向（不限制

垂直次大方向会和最大方向无限接近), 作为新坐标系的第二个轴; 依此类推, 直到找出了 K 个, 然后把原来的数据用新的坐标系进行表示 (也就是进行投影), 就得到了降维后的数据。

3. PCA计算流程

1) 去中心化

按列计算数据集 X 的均值, 然后对每一列进行归一化处理, 即每个特征值减去其对应的均值, 使得每个特征的均值变为0。这一步是为了确保数据在原点附近分布, 便于后续计算。

2) 计算协方差矩阵

基于预处理后的数据, 计算其协方差矩阵 C 。协方差矩阵是一个 $n \times n$ 的矩阵, 其中每个元素表示两个特征之间的协方差。

3) 特征值分解

对协方差矩阵 C 进行特征值分解, 得到特征值 λ 和对应的特征向量 v 。特征值表示数据在特征向量方向上的方差大小, 而特征向量则表示数据的分布方向。将特征值从大到小进行排序, 并对应地调整特征向量的顺序。

4) 投影数据得到降维后的数据

根据降维需求, 选择前 K 个特征值对应的特征向量作为主成分, 这些主成分将构成新的特征空间。将预处理后的数据投影到选定的主成分上, 得到降维后的数据。这一步是通过计算原始数据矩阵与主成分矩阵的乘积来实现的。

6.1.2 K-means

1. K-means算法背景

随着大数据时代的到来, 数据量的激增带来了数据挖掘和分析的需求。聚类分析作为数据挖掘中的一项重要技术, 旨在将数据集中的对象根据相似性分组多个簇, 使得同一个簇内的对象之间的相似度高, 而不同簇之间的对象相似度低。K-means算法是聚类分析中最著名且使用最广泛的算法之一, 它简单、高效, 尤其适用于大规模数据集。

2. K-means算法原理

K均值聚类 (K-means) 算法基于距离度量, 将数据点划分为 K 个簇, 每个簇由一个聚类中心 (质心) 来代表。算法的核心思想是最小化簇内距离的总和, 即每个点到其簇中心的距离平方和。通过迭代地调整簇中心的位置, 算法逐步找到局部最优解。

举一个简单的例子讲解一下。

本例中将 K 设置为2, 在图6-4中包括6个数据点和随机生成的2个中心簇, 分别计算6个数据点到2个中心簇的距离, 对于数据点 $x=(x_1, x_2, \dots, x_n)$ 和聚类中心 $c=(c_1, c_2, \dots, c_n)$, 它们之间的欧几里得距离 d 计算公式是:

$$d(x, c) = \sqrt{(x_1 - c_1)^2 + (x_2 - c_2)^2 + \dots + (x_n - c_n)^2} \quad (6-2)$$

在图6-5中标出距离两个中心簇更近的数据点, 接下来更新中心簇的位置, 更新中心簇的算法是将距离目标簇更近的数据点的各个对应维度数值相加取平均值为新的中心簇的位置, 如图6-6所示。接下来继续划分距离中心簇近的数据点, 计算算法和图6-4的计算过程一样, 采用欧几里得距离, 得出图6-7, 之后继续更新中心簇的位置得图6-8。最后就可以明显看出左边四个数据点属于第一个簇, 右边两个数据点属于第二个簇。

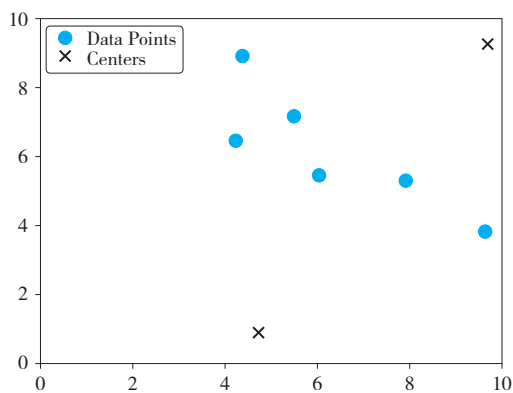


图 6-4 初始状态

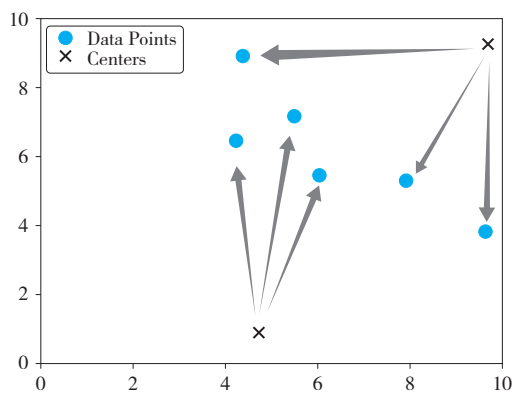


图 6-5 分配数据点到簇

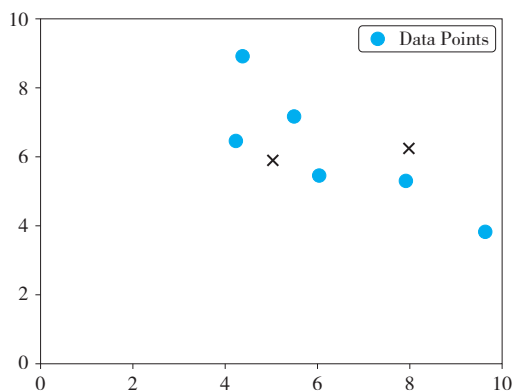


图 6-6 更新簇中心

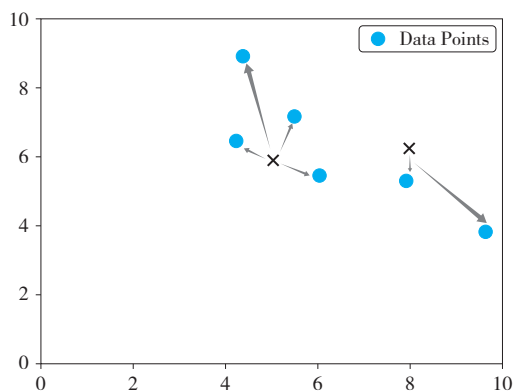


图 6-7 再次分配数据点

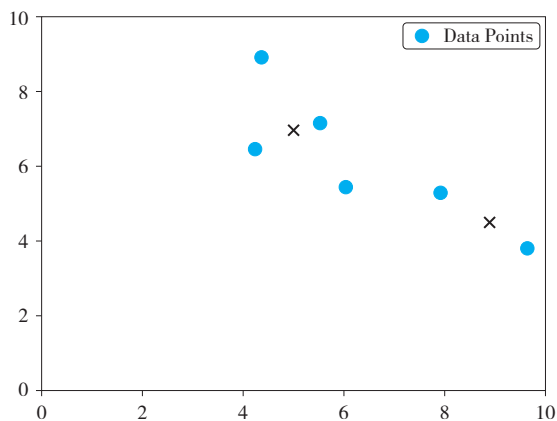


图 6-8 收敛状态

K-Means主要缺陷都和初始值有关： K 是事先给定的，这个 K 值的选定是非常难以估计的。很多时候，事先并不知道给定的数据集应该分成多少个类别才最合适。

3. K-means算法步骤

(1) 初始化。

选择初始聚类中心：从数据集中随机选择 K 个对象作为初始聚类中心。

(2) 分配。

计算距离：对于数据集中的每个对象，计算它到每个聚类中心的距离。通常使用欧几里得距离，但也可以采用其他距离度量。

划分对象：根据计算出的距离，将每个对象分配给最近的聚类中心，从而形成 K 个簇。

(3) 更新。

重新计算聚类中心：对于每个簇，计算簇中所有对象的平均值（质心），并用这个平均值更新聚类中心的位置。

(4) 迭代。

计算标准测度函数：通常使用簇内误差平方和（SSE）作为标准测度函数，即每个对象到其聚类中心的距离平方和。

检查停止条件：重复步骤（2）和（3），直到满足停止条件，如达到预设的最大迭代次数或聚类中心的变化小于某个阈值。如果没有达到停止条件，则继续迭代。

(5) 确定最优聚类中心。

收敛：当算法达到停止条件时，算法收敛，此时的聚类中心被认为是最优的聚类中心。

6.2 案例描述

6.2.1 案例背景介绍

在当今数据爆炸式增长的时代，大量数据以无监督的形式存在，这些数据没有预先标注的类别或目标变量。无监督学习作为机器学习的重要分支，能够从这些未标注的数据中发现潜在的模式、结构和特征，为数据挖掘和知识发现提供了强大的工具。本案例聚焦无监督学习中的两种经典算法：主成分分析和 K 均值聚类，通过实际案例探讨无监督学习在数据降维和聚类分析中的应用。

在计算机视觉领域，人脸图像包含大量的数据信息（像素），直接处理这些高维数据不仅计算成本高，而且效率低下。使用主成分分析，可以有效降低人脸图像的维度，提取出最有信息量的特征，从而简化人脸识别过程。

在数据挖掘和信息检索等领域，面对海量的未标注数据，需要一种有效的方法来发现数据的内在结构和模式。 K 均值聚类作为一种广泛使用的聚类算法，通过迭代寻找数据簇的最佳划分，能够将相似的数据点聚集在一起，为数据的组织和分析提供了有力支持。

6.2.2 案例技术分析

本案例将采用主成分分析和 K 均值聚类这两种典型的无监督学习技术来处理不同类型的数据集，深入理解无监督学习的核心思想和应用方法。

1. 主成分分析

技术优势：能够有效降低数据维度，减少计算复杂度；去除数据中的冗余信息和噪声，提高数据质量；保留数据的主要特征和变化模式，便于后续分析和处理。

应用场景：适用于高维数据的降维处理，如工业生产中的传感器数据、图像数据等；适用于数据可视化，将高维数据投影到低维空间进行展示；作为其他机器学习算法的预处理步骤，提高模型的性能和效率。

2. K均值聚类

技术优势：算法简单、高效，易于实现和理解；适用于大规模数据集的聚类分析；对球形簇数据的聚类效果较好。

应用场景：适用于数据挖掘中的模式识别和数据分类，如客户细分、文本聚类等；在图像处理中，用于图像分割和特征提取；在生物信息学中，用于基因表达数据的聚类分析。

6.3 设计与实现

6.3.1 使用 PCA 对半导体数据进行降维处理

1. 案例任务

本案例围绕高维工业数据的降维分析展开，核心学习目标包括：掌握高维工业数据预处理与特征工程的标准化流程；理解 PCA 在数据压缩与关键特征提取中的核心作用；评估降维技术对工业数据分析的优化效果；探索降维特征对机器学习模型泛化能力的提升机制。为达成上述目标，需完成以下具体任务：首先对原始半导体数据集开展预处理工作，重点处理数据中的缺失值；其次应用 PCA 技术实现数据维度降低，并深入分析提取出的主要成分；接着通过可视化方法直观展示降维前后的数据差异，完成降维效果的评估；最后结合分类、预测等典型机器学习任务，探讨 PCA 降维对后续模型性能的潜在影响。

2. 开发环境

硬件环境：标准PC或服务器。

操作系统：Windows, Linux或Mac OS。

编程环境：Python 3.x。

主要依赖库：NumPy, Matplotlib, Scikit-learn。

数据集：半导体制造过程中的数据集，包含590个特征，每个特征对应设备的一个测量维度。

3. 实现步骤

(1) 加载数据集。首先需要加载半导体数据集。在这一步中，读取文件内容，并将其转换成矩阵格式，为进一步分析准备数据。

加载数据集：

```
from numpy import *  
def loadDataSet(fileName, delim='\t'):  
    fr = open(fileName)  
    stringArr = [line.strip().split(delim) for line in fr.readlines()]  
    datArr = [list(map(float, line)) for line in stringArr]  
    return mat(datArr)  
dataMat = loadDataSet('secom.data', ' ')
```

图6-9中可见dataMat中的前20个数据。

```
In [5]: dataMat[:20]
Out[5]: matrix([[3.0309300e+03, 2.5640000e+03, 2.1877333e+03, ..., nan,
nan, nan],
[3.0957800e+03, 2.4651400e+03, 2.2304222e+03, ..., 2.0100000e-02,
6.0000000e-03, 2.0820450e+02],
[2.9326100e+03, 2.5599400e+03, 2.1864111e+03, ..., 4.8400000e-02,
1.4800000e-02, 8.2860200e+01],
...,
[3.0327300e+03, 2.5177900e+03, 2.2702556e+03, ..., 1.5100000e-02,
4.2000000e-03, 8.2098900e+01],
[3.0403400e+03, 2.5011600e+03, 2.2073889e+03, ..., 1.5100000e-02,
4.2000000e-03, 8.2098900e+01],
[2.9883000e+03, 2.5190500e+03, 2.2088556e+03, ..., 1.0800000e-02,
3.2000000e-03, 4.7158600e+01]])
```

图 6-9 dataMat 中的前 20 个数据

(2) 预处理数据。在数据预处理步骤中，主要任务是处理数据集中的缺失值。使用列的均值来替换NaN值，以确保数据的完整性，为PCA分析做准备。

预处理数据，替换NaN值为该列的均值：

```
def replaceNanWithMean():
    datMat = loadDataSet('secom.data', ' ')
    numFeat = shape(datMat)[1]
    for i in range(numFeat):
        # 对 value 不为 NaN 的求均值
        # 下面函数中属性返回矩阵基于的数组
        meanVal = mean(datMat[nonzero(~isnan(datMat[:, i].A))[0], i])
        # 将 value 为 NaN 的值赋值为均值
        datMat[nonzero(isnan(datMat[:, i].A))[0], i] = meanVal
    return datMat

dataMat = replaceNanWithMean()
```

如图6-10所示，经过预处理数据，数据中的NaN值已经被替换。

```
In [7]: dataMat[:20]
Out[7]: matrix([[3.0309300e+03, 2.5640000e+03, 2.1877333e+03, ...,
1.6474904e-02, 5.2833333e-03, 9.9670066e+01],
[3.0957800e+03, 2.4651400e+03, 2.2304222e+03, ...,
2.0100000e-02, 6.0000000e-03, 2.0820450e+02],
[2.9326100e+03, 2.5599400e+03, 2.1864111e+03, ...,
4.8400000e-02, 1.4800000e-02, 8.2860200e+01],
...,
[3.0327300e+03, 2.5177900e+03, 2.2702556e+03, ...,
1.5100000e-02, 4.2000000e-03, 8.2098900e+01],
[3.0403400e+03, 2.5011600e+03, 2.2073889e+03, ...,
1.5100000e-02, 4.2000000e-03, 8.2098900e+01],
[2.9883000e+03, 2.5190500e+03, 2.2088556e+03, ...,
1.0800000e-02, 3.2000000e-03, 4.7158600e+01]])
```

图 6-10 替换 NaN 值为列均值

(3) 从数据中减去平均值。对数据进行中心化处理，即每个维度减去其平均值，使得数据在每个维度上的均值为零。这一步是PCA的关键步骤，旨在消除原点选择对数据分析结果的影响。

减去均值：

```
meanVals = mean(dataMat, axis=0)
meanRemoved = dataMat - meanVals
```

如图6-11所示，已减去均值。

In [10]:	meanRemoved
Out[10]:	matrix([[1.64771044e+01, 6.81497692e+01, -1.28140177e+01, ..., 2.7755756e-17, 1.99493200e-17, 1.70530257e-13], [8.13271044e+01, -3.07102308e+01, 2.98748823e+01, ..., 3.62509579e-03, 7.16666667e-04, 1.08534434e+02], [-8.18428956e+01, 6.40897692e+01, -1.41362177e+01, ..., 3.19250958e-02, 9.51666667e-03, -1.68098663e+01], ..., [-3.56428956e+01, -1.16070231e+02, 5.75268229e+00, ..., -7.87490421e-03, -2.78333333e-03, -5.61469663e+01], [-1.19532896e+02, 3.61597692e+01, -2.35140177e+01, ..., 8.02509579e-03, 2.21666667e-03, -6.17596635e+00], [-6.95328956e+01, -4.50902308e+01, -5.10291771e+00, ..., 2.74664915e-04, 7.62932222e-04, 2.61149927e+01]])

图 6-11 从数据中减去均值

(4) 计算协方差矩阵。使用中心化后的数据计算协方差矩阵，协方差矩阵揭示了数据集中各个特征之间的线性关系。

计算协方差矩阵：

```
covMat = cov(meanRemoved, rowvar=0)
```

(5) 计算特征值和特征向量。计算协方差矩阵的特征值和特征向量。特征向量代表数据中的主成分方向，而特征值的大小反映了各主成分的重要性。

计算特征值和特征向量：

```
eigVals, eigVects = linalg.eig(mat(covMat))
```

(6) 选择主成分。根据特征值的大小，选择贡献最大的若干个特征向量作为主成分。这些主成分将用于构建新的、降维后的特征空间。

选择主成分：

```
eigValInd = argsort(eigVals)
topNfeat = 20
eigValInd = eigValInd[:-(topNfeat+1):-1]
redEigVects = eigVects[:, eigValInd]
```

(7) 将数据转换到新空间。使用选择的主成分将原始数据转换到新的特征空间，实现降维。

将数据转换到新空间：

```
lowDDataMat = meanRemoved * redEigVects
reconMat = (lowDDataMat * redEigVects.T) + meanVals
```

(8) 分析结果。分析PCA的结果，可以通过计算每个主成分的方差占比来了解每个主成分的重要性。

```
def analyse_data(dataMat):
    meanVals = mean(dataMat, axis=0)
    meanRemoved = dataMat - meanVals
    covMat = cov(meanRemoved, rowvar=0)
    eigvals, eigVects = linalg.eig(mat(covMat))
    eigValInd = argsort(eigvals)
```

```

topNfeat = 20
eigValInd = eigValInd[:-(topNfeat+1):-1]
cov_all_score = float(sum(eigvals))
sum_cov_score = 0
for i in range(0, len(eigValInd)):
    line_cov_score = float(eigvals[eigValInd[i]])
    sum_cov_score += line_cov_score
    print("主成分: {}, 方差占比: {}, 累积方差占比: {}".format(i+1, '2.0f', line_
cov_score/cov_all_score*100, '4.2f', (sum_cov_score/cov_all_score*100, '4.1f'))))
analyse_data(dataMat)

```

分析结果如图6-12所示。

```

主成分: 1, 方差占比: 2.0f59.25405798334101, 累积方差占比: 4.2f(59.25405798334101, '4.1f')
主成分: 2, 方差占比: 2.0f24.123818869605024, 累积方差占比: 4.2f(83.37787685294603, '4.1f')
主成分: 3, 方差占比: 2.0f9.150013588650662, 累积方差占比: 4.2f(92.5278904415967, '4.1f')
主成分: 4, 方差占比: 2.0f2.300578517305427, 累积方差占比: 4.2f(94.8284689589021, '4.1f')
主成分: 5, 方差占比: 2.0f1.4591923458304508, 累积方差占比: 4.2f(96.28766130473258, '4.1f')
主成分: 6, 方差占比: 2.0f0.5188175327846472, 累积方差占比: 4.2f(96.80647883751723, '4.1f')
主成分: 7, 方差占比: 2.0f0.3226580948767247, 累积方差占比: 4.2f(97.12913693239396, '4.1f')
主成分: 8, 方差占比: 2.0f0.31467665356345587, 累积方差占比: 4.2f(97.44381358595741, '4.1f')
主成分: 9, 方差占比: 2.0f0.26307953264531386, 累积方差占比: 4.2f(97.70689311860272, '4.1f')
主成分: 10, 方差占比: 2.0f0.23130665828884367, 累积方差占比: 4.2f(97.93819977689158, '4.1f')
主成分: 11, 方差占比: 2.0f0.21753457834906356, 累积方差占比: 4.2f(98.15573435524064, '4.1f')
主成分: 12, 方差占比: 2.0f0.20728199528122415, 累积方差占比: 4.2f(98.36301635052187, '4.1f')
主成分: 13, 方差占比: 2.0f0.1690837694790774, 累积方差占比: 4.2f(98.53210012000095, '4.1f')
主成分: 14, 方差占比: 2.0f0.1255906621886416, 累积方差占比: 4.2f(98.6576907821896, '4.1f')
主成分: 15, 方差占比: 2.0f0.12035340182520511, 累积方差占比: 4.2f(98.7780441840148, '4.1f')
主成分: 16, 方差占比: 2.0f0.11409210162539687, 累积方差占比: 4.2f(98.8921362856402, '4.1f')
主成分: 17, 方差占比: 2.0f0.11111541175048761, 累积方差占比: 4.2f(99.0032516973907, '4.1f')
主成分: 18, 方差占比: 2.0f0.09245814782889004, 累积方差占比: 4.2f(99.09570984521959, '4.1f')
主成分: 19, 方差占比: 2.0f0.09050319036640793, 累积方差占比: 4.2f(99.18621303558601, '4.1f')
主成分: 20, 方差占比: 2.0f0.08614470068195862, 累积方差占比: 4.2f(99.27235773626796, '4.1f')

```

图 6-12 分析结果

(9) 可视化结果。最后，可视化原始数据和降维后的数据，这有助于理解PCA降维的效果，如图6-13所示。

```

# 可视化结果
import matplotlib.pyplot as plt
def show_picture(dataMat, reconMat):
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax.scatter(dataMat[:, 0].flatten().A[0], dataMat[:, 1].flatten().A[0], marker='^',
s=90)
    ax.scatter(reconMat[:, 0].flatten().A[0], reconMat[:, 1].flatten().A[0], marker='o',
s=50, c='red')
    plt.show()
show_picture(dataMat, reconMat)

```

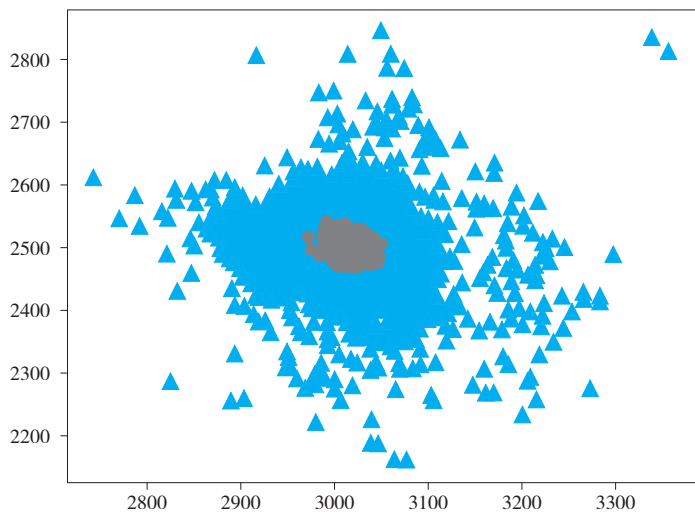


图 6-13 可视化结果图

（10）完成案例并撰写报告。在案例报告中总结PCA的结果，包括主成分的重要性、降维的效果，以及降维对后续数据分析可能产生的影响。

这些步骤概括了使用PCA进行数据降维的整个流程，并与案例代码相结合，形成了完整的案例设计。

根据图6-13，其中深色三角形代表原始数据点，浅色区域表示经PCA变换后数据点的密集区域。

4. PCA结果分析

数据分布：图6-13中数据分布似乎显示出一定程度的聚集趋势，这表明在降维后的空间中，可能存在可以区分的数据簇或者组。

离群点：有些数据点分布在密集区域之外，这些可能是离群点或者异常值。

数据密度：红色区域集中体现了数据点的高密度区域，这可能是数据的主要变量集中的区域。

探讨和总结：

讨论PCA降维的效果与潜在影响。

探讨PCA对后续模型训练的潜在好处，如提高模型训练速度、减少过拟合等。

6.3.2 PCA 的人脸识别算法

1. 案例任务

人脸识别是计算机视觉领域中一个重要的应用，涉及图像处理、机器学习等多个技术领域。由于人脸图像包含大量的数据信息（像素），直接处理这些高维数据不仅计算成本高，而且效率低下。使用主成分分析（PCA），也被称为特征脸方法，可以有效降低人脸图像的维度，提取出最有信息量的特征，从而简化人脸识别过程。

2. 开发环境

硬件环境：标准PC或服务器。

操作系统：Windows，Linux 或Mac OS.

编程环境：Python 3.x。

主要依赖库：NumPy, opencv-python, Matplotlib, Scikit-learn, pillow。

数据集：ORL人脸数据集（OlivettiFaces），包含40个不同人的脸图像，每人10张，共400张。

图像规格：92×112像素灰度图，表情/姿态略有变化，无明显遮挡。

3. 实现步骤

PCA人脸识别算法主要通过以下几个步骤实现：

（1）**图像预处理**：将所有的人脸图像调整为统一的大小，通常是灰度图像，确保每个图像的像素数目相同。

（2）**构建数据矩阵**：将预处理后的每个图像转换为一个向量，所有的图像向量组成一个数据矩阵。

（3）**中心化处理**：计算所有图像向量的平均向量，并用每个图像向量减去这个平均向量，以确保数据矩阵是中心化的。

（4）**计算协方差矩阵**：在中心化的数据基础上计算协方差矩阵，用于分析各像素间的相关性。

（5）**特征值和特征向量**：对协方差矩阵进行特征分解，得到特征值和对应的特征向量（特征脸）。

（6）**选择主成分**：根据特征值的大小选择最重要的特征向量，这些特征向量定义了人脸的主要特征。

（7）**将图像投影到特征空间**：使用选定的特征向量将原始图像向量转换到低维的特征空间中。

（8）**分类与识别**：在特征空间中，可以使用简单的距离度量（如欧氏距离）来进行人脸识别，或者结合机器学习算法进行更复杂的分类任务。

4. 案例代码

首先定义一个函数用于将人脸图像矢量化为一个向量，向量的大小与图片的像素有关，代码如下：

```
import cv2
import numpy as np
# 图片矢量化
def img2vector(image):
    img = cv2.imread(image, 0) # 读取图片
    rows, cols = img.shape # 获取图片的像素
    imgVector = np.zeros((1, rows * cols)) # 初始值均设置为 0，大小就是图片像素的大小
    imgVector = np.reshape(img, (1, rows * cols)) # 使用 imgVector 变量作为一个向量存储图片矢量化信息
    return imgVector
```

接下来定义一个函数用来选取训练图片，并对每张图片进行前面定义过的矢量化处理。

```
def load_orl(k): # 参数 k 代表选择 k 张图片作为训练图片使用
    '''
    对训练数据集进行数组初始化，用 0 填充，每张图片尺寸都定为 112*92，
    现在共有 40 个人，每个人都选择 k 张，则整个训练集大小为 40*k, 112*92
    '''
    train_face = np.zeros((40 * k, 112 * 92))
    train_label = np.zeros(40 * k) # [0,0,...,0] (共 40*k 个 0)
    test_face = np.zeros((40 * (10 - k), 112 * 92))
    test_label = np.zeros(40 * (10 - k))
```



```

    orlpath = 'ORL' # 存放图片的路径
    # sample=random.sample(range(10),k)# 每个人都有的 10 张照片中, 随机选取 k 张作为训练样本
    # (10 个里面随机选取 K 个成为一个列表)
    sample = random.permutation(10) + 1 # 随机排序 1-10 (0-9)+1
    for i in range(40): # 共有 40 个人
        people_num = i + 1
        for j in range(10): # 每个人都有 10 张照片
            image = orlpath + '/s' + str(people_num) + '/' + str(sample[j]) + '.jpg'
            # 读取图片并进行矢量化
            img = img2vector(image)
            if j < k:
                # 构成训练集
                train_face[i * k + j, :] = img
                train_label[i * k + j] = people_num
            else:
                # 构成测试集
                test_face[i * (10 - k) + (j - k), :] = img
                test_label[i * (10 - k) + (j - k)] = people_num

    return train_face, train_label, test_face, test_label

```

前期将所有训练图片矢量化之后, 开始进行PCA算法的降维操作。

```

def PCA(data, r):# 参数 r 代表降低到 r 维
    data = np.float32(np.mat(data))
    rows, cols = np.shape(data)
    data_mean = np.mean(data, 0) # 对列求平均值
    A = data - np.tile(data_mean, (rows, 1)) # 将所有样例减去对应均值得到 A
    C = A * A.T # 得到协方差矩阵
    D, V = np.linalg.eig(C) # 求协方差矩阵的特征值和特征向量
    V_r = V[:, 0:r] # 按列取前 r 个特征向量
    V_r = A.T * V_r # 小矩阵特征向量向大矩阵特征向量过渡
    for i in range(r):
        V_r[:, i] = V_r[:, i] / np.linalg.norm(V_r[:, i]) # 特征向量归一化

    final_data = A * V_r
    return final_data, data_mean, V_r

```

最后进行初次训练, 随机选取每个人物的五张图片作为训练图片使用。将降低的维数设定为10维, 查看训练效果如何。

```

def face_recongize():
    # 对每一个人随机选取 5 张照片作为训练数据
    train_face, train_label, test_face, test_label = load_orl(5) # 随机选择每个人物的 5 张

```

图片作为训练数据

```
x_value = []
y_value = []
# 将图片降维到 10 维
data_train_new, data_mean, V_r = PCA(train_face, 10)
num_train = data_train_new.shape[0] # 训练脸总数
num_test = test_face.shape[0] # 测试脸总数
temp_face = test_face - np.tile(data_mean, (num_test, 1))
data_test_new = temp_face * V_r # 得到测试脸在特征向量下的数据
data_test_new = np.array(data_test_new) # mat change to array
data_train_new = np.array(data_train_new)
true_num = 0
for i in range(num_test):
    testFace = data_test_new[i, :]
    diffMat = data_train_new - np.tile(testFace, (num_train, 1)) # 训练数据与测试脸之
间的距离
    sqDiffMat = diffMat ** 2
    sqDistances = sqDiffMat.sum(axis=1) # 按行求和
    sortedDistIndicies = sqDistances.argsort() # 对向量从小到大排序, 使用的是索引值, 得
到一个向量
    indexMin = sortedDistIndicies[0] # 距离最近的索引
    if train_label[indexMin] == test_label[i]:
        true_num += 1
    else:
        pass
accuracy = float(true_num) / num_test
x_value.append(5)
y_value.append(round(accuracy, 2))

print(' 当对每个人随机选择 %d 张照片降低至 %d 维进行训练时, The classify accuracy is: %.2f%%'
      % (5, 10, accuracy * 100))
```

最终训练得到的结果如下, 如图6-14所示。

当降维到 10 时

```
当每个人选择 1 张照片进行训练时, The classify accuracy is: 58.61%
当每个人选择 2 张照片进行训练时, The classify accuracy is: 74.06%
当每个人选择 3 张照片进行训练时, The classify accuracy is: 86.43%
当每个人选择 4 张照片进行训练时, The classify accuracy is: 85.83%
当每个人选择 5 张照片进行训练时, The classify accuracy is: 90.00%
当每个人选择 6 张照片进行训练时, The classify accuracy is: 92.50%
当每个人选择 7 张照片进行训练时, The classify accuracy is: 95.00%
当每个人选择 8 张照片进行训练时, The classify accuracy is: 88.75%
当每个人选择 9 张照片进行训练时, The classify accuracy is: 100.00%
```

当降维到 20 时

当每个人选择 1 张照片进行训练时, The classify accuracy is: 65.56%
当每个人选择 2 张照片进行训练时, The classify accuracy is: 77.50%
当每个人选择 3 张照片进行训练时, The classify accuracy is: 86.79%
当每个人选择 4 张照片进行训练时, The classify accuracy is: 88.75%
当每个人选择 5 张照片进行训练时, The classify accuracy is: 93.50%
当每个人选择 6 张照片进行训练时, The classify accuracy is: 96.88%
当每个人选择 7 张照片进行训练时, The classify accuracy is: 97.50%
当每个人选择 8 张照片进行训练时, The classify accuracy is: 96.25%
当每个人选择 9 张照片进行训练时, The classify accuracy is: 100.00%

当降维到 30 时

当每个人选择 1 张照片进行训练时, The classify accuracy is: 64.44%
当每个人选择 2 张照片进行训练时, The classify accuracy is: 83.75%
当每个人选择 3 张照片进行训练时, The classify accuracy is: 85.36%
当每个人选择 4 张照片进行训练时, The classify accuracy is: 89.58%
当每个人选择 5 张照片进行训练时, The classify accuracy is: 90.50%
当每个人选择 6 张照片进行训练时, The classify accuracy is: 96.25%
当每个人选择 7 张照片进行训练时, The classify accuracy is: 95.00%
当每个人选择 8 张照片进行训练时, The classify accuracy is: 96.25%
当每个人选择 9 张照片进行训练时, The classify accuracy is: 95.00%

当降维到 40 时

当每个人选择 1 张照片进行训练时, The classify accuracy is: 70.83%
当每个人选择 2 张照片进行训练时, The classify accuracy is: 77.81%
当每个人选择 3 张照片进行训练时, The classify accuracy is: 85.71%
当每个人选择 4 张照片进行训练时, The classify accuracy is: 92.92%
当每个人选择 5 张照片进行训练时, The classify accuracy is: 96.00%
当每个人选择 6 张照片进行训练时, The classify accuracy is: 93.12%
当每个人选择 7 张照片进行训练时, The classify accuracy is: 95.83%
当每个人选择 8 张照片进行训练时, The classify accuracy is: 97.50%
当每个人选择 9 张照片进行训练时, The classify accuracy is: 97.50%

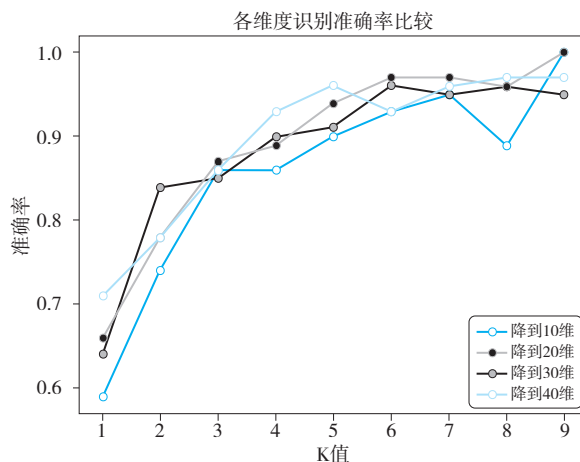


图 6-14 降维结果图

5. 代码运行结果分析

降维的影响：代码运行结果表明，随着降维维度的增加，从10维到40维，分类的准确率整体呈上升趋势。这说明在所选择的维度范围内，更高的维度能够保留更多的信息，有利于提高识别的准确性。

PCA的特性：通过对协方差矩阵的特征值分解来找到数据的主要变化方向，PCA提供了一种有效的方式来减少数据集的维数，同时保留最重要的信息。特征向量（或特征脸）对应了数据集中最大方差的方向，这些方向是进行人脸识别最关键的特征。

准确率和维度的平衡：实验数据表明，随着维度的增加，准确率提高，但维度增加到一定程度后，准确率的提升趋于平缓。这可能表明在达到某个维数后，进一步增加维数所增加的信息量对分类的帮助不大。因此，在实际应用中需要找到一个维数和准确率的平衡点。

综上所述，PCA是一种强大的降维工具，对于人脸识别等高维数据处理任务非常有效。然而，为了获得最佳的识别效果，需要通过实验来确定最佳的维数，并且需要足够的训练数据来保证模型的泛化能力。在实际应用中，这些代码运行结果将有助于指导如何选择合适的训练数据量和PCA降维的维度。

6.3.3 使用 K-means 对数据集进行聚类分析

1. 案例任务

本案例围绕经典聚类算法K-means 展开实践，旨在系统掌握聚类算法的完整应用流程。具体任务包括：生成模拟数据集并完成聚类分析，处理非等距数据场景下的聚类问题，对经过过滤处理的数据进行聚类，以及对各类聚类结果开展效果分析与可视化展示，从而全面理解K-means在不同数据分布与预处理条件下的聚类性能与适用特点。

2. 开发环境

硬件环境：标准PC或服务器。

操作系统：Windows，Linux或Mac OS.

编程环境：Python 3.x。

主要依赖库：NumPy, Pandas, Matplotlib, Scikit-learn, scipy。

3. 实现步骤

(1) 导入必要的库。

```
# coding:utf-8
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

(2) 生成模拟数据并进行聚类。

使用make_blobs函数生成随机数据点（x），并为这些数据点分配类别标签（y）。

应用 K-means 算法（KMeans（n_clusters=3））对数据进行聚类，这里将数据分为3个簇。

打印原始数据点和预测的类别标签。

```
# 建立 12*12 英寸新的图像
plt.figure(figsize=(12, 12))
n_samples = 150
```

```

random_state = 170
'''
    make_blobs 函数是为聚类产生数据集，产生一个数据集和相应的标签
    n_samples: 表示数据样本点个数，默认值 100
    n_features: 表示数据的维度、特征，默认值是 2
    centers: 产生数据的中心点，默认值 3 个
    shuffle : 洗乱，默认值是 True
    random_state: 官网解释是随机生成器的种子
'''
# x 返回的是向量化的数据点，y 返回的是对应数据的类别号
x, y = make_blobs(n_samples=n_samples, random_state=random_state)
print('x=', x, type(x), 'y=', y, type(y))
# 使用 KMeans 去聚类，返回聚好的类别集合，n_clusters 聚合成几类
y_pred = KMeans(n_clusters=3).fit_predict(x)
print("y_pred : ", y_pred)

```

(3) 绘制散点图。

使用 Matplotlib 的 subplot 和 scatter 函数绘制原始数据点的散点图，并根据 K-means 聚类结果为不同簇的数据点着色。

```

# subplot 绘制多个子图，221 等价于 2,2,1，表示两行两列的子图中的第一个
plt.subplot(221)
# scatter 绘制散点图，c 指定颜色
plt.scatter(x[:, 0], x[:, 1], c=y_pred)
plt.title("kmeans01")

```

(4) 处理非等距数据的聚类。

通过一个变换矩阵 transformation 对数据进行变换，模拟非等距（非球形）数据的聚类。

```

transformation = [[0.60834549, -0.63667341],
                  [-
                    0.40887718, 0.85253229]]

# numpy.dot 矩阵相乘
# a1= [[1,2]
#      [3,4]
#      [5,6]]
# a2= [[10,20]
#      [30,40]]
# a1*a2 = [[1*10+2*30, 1*20+2*40]
#          [3*10+4*30, 3*20+4*40]
#          [5*10+5*30, 6*20+6*40]
#          ]
X_aniso = np.dot(x, transformation)

```

(5) 处理过滤后的数据聚类。

从原始数据集中筛选特定类别的数据点，形成一个新的数据集 X_filtered。对过滤后的数据集应用

K-means 聚类。

```
y_pred = KMeans(n_clusters=3).fit_predict(X_aniso)
plt.subplot(222)
# # s 设置点的大小,s=8
plt.scatter(X_aniso[:, 0], X_aniso[:, 1], c=y_pred)
plt.title("kmeans02")

# vstack 是合并矩阵,将 y=0 类别的取出 500 行, y=1 类别的取出 100 行, y=2 类别的取出 10 行
X_filtered = np.vstack((x[y == 0][:500], x[y == 1][:100], x[y == 2][:200]))
y_pred = KMeans(n_clusters=3, random_state=random_state).fit_predict(X_filtered)
plt.subplot(223)
plt.scatter(X_filtered[:, 0], X_filtered[:, 1], c=y_pred)
plt.title("kmeans03")
```

(6) 读取外部数据文件并进行聚类。

```
dataMat = []
fr = open("testSet.txt", "rb")
for line in fr.readlines():
    if line.decode("utf-8").strip() != "":
        curLine = line.decode("utf-8").strip().split('\t')
        fltLine = list(map(float, curLine))
        dataMat.append(fltLine)
dataArray = np.array(dataMat)
print("dataArray type = %s"% type(dataArray))
# 调用 Scikitlearn 中的 KMeans
# KMeans 中参数 init='k-means++' 默认就是 k-means++, 如果设置为 'random' 是随机找中心点
y_pred = KMeans(n_clusters=5).fit_predict(dataArray)
```

(7) 保存和显示图像。

```
plt.subplot(224)
plt.scatter(dataArray[:, 0], dataArray[:, 1], c=y_pred)
plt.title("kmeans04")
plt.savefig("./kmeans.png")
plt.show()
```

图6-15显示了用K-means算法聚类后的二维数据点分布。可以看到,共有四个不同的聚类,每个聚类用不同颜色的点表示。从聚类分布来看,算法似乎已经较好地识别出了数据点之间的差异,并将其分组。例如:

第一个图中(kmeans01)黄色和青色点集中在左上方和右下方,显示了两个明显分离的簇。

第二个图中(kmeans02)紫色点形成了一个延伸的簇,黄色和青色点分布在其两侧,这可能表示三个不同的主题或类别。

第三个和第四个图(kmeans03和kmeans04)分别展示了两个和四个较为分散的簇,反映了数据的不同分布情况。

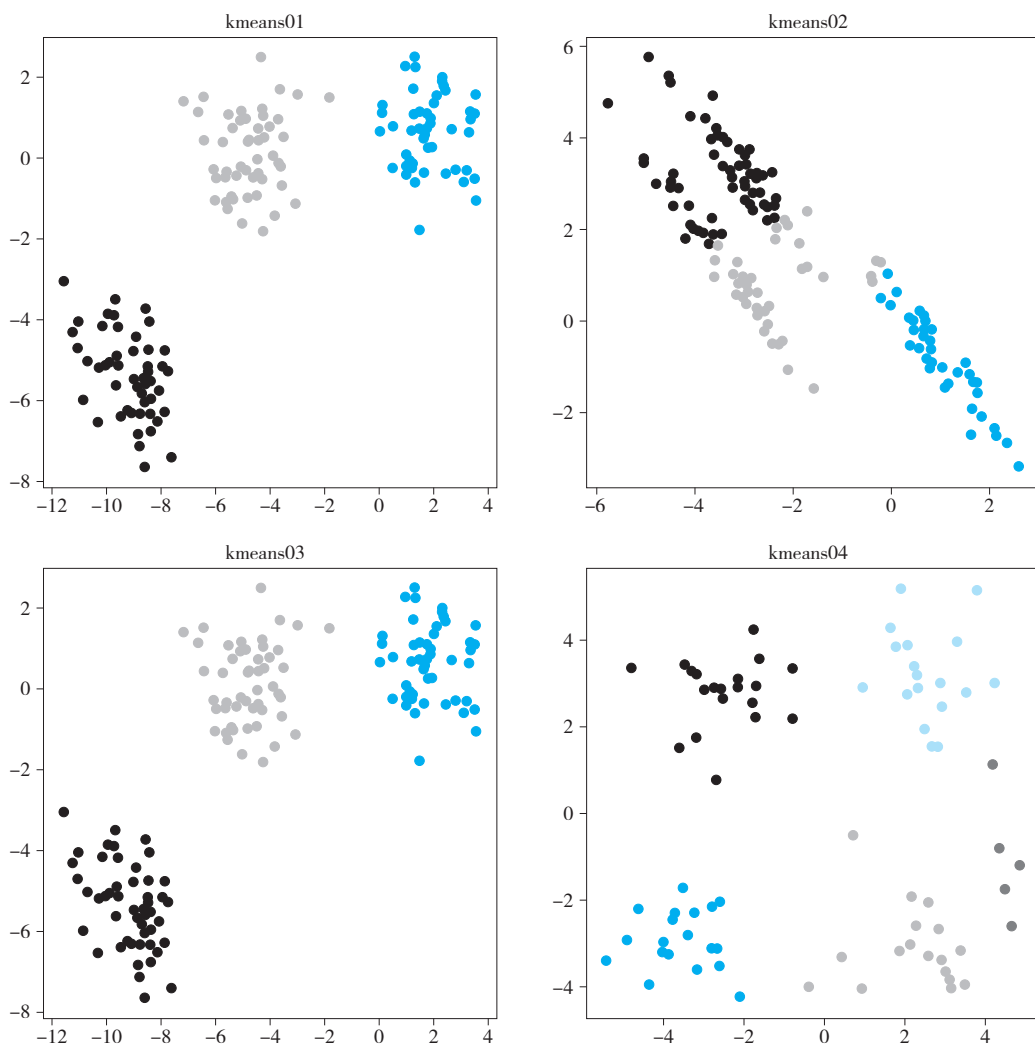


图 6-15 基于 K-means 算法的数据聚类可视化图

通过聚类结果可视化，可以初步判断K-means 聚类算法对不同数据集进行聚类分析时的效果很好。

总 结

本章深入探讨了无监督学习中的两个核心算法：主成分分析和K均值聚类，并通过一系列案例来理解它们的基本原理和应用。

主成分分析：我们首先了解了PCA的数学原理，它是一种通过线性变换将数据集从高维降至低维的技术，同时尽可能保留原始数据的重要信息。PCA的计算流程包括去中心化、计算协方差矩阵、特征值分解，以及选择主成分并投影数据到新的特征空间。通过半导体数据降维案例和人脸识别算法案例，我

们学习了如何在实际数据集上应用PCA进行特征提取和模式识别，并分析了降维对数据压缩和数据挖掘的影响。

K均值聚类：接着，我们学习了K-means聚类算法，这是一种基于距离度量的聚类方法，通过迭代调整簇中心的位置来最小化簇内距离的总和。此次案例展示了如何使用 K-means 算法对不同类型的数据集进行聚类分析，并通过可视化手段展示聚类结果。这有助于理解 K-means 算法在实际应用中的效果和性能。

习 题

1. 使用PCA对手写数字识别数据集进行降维处理，分析并解释降维效果。步骤如下：

- (1) 加载数据集并进行预处理。
- (2) 应用PCA进行数据降维，探索不同降维维度对数据集的影响。
- (3) 使用降维后的数据进行分类任务，并评估分类器的性能。
- (4) 可视化降维效果，比较降维前后的数据分布。

可以通过如下代码加载数据集：

```
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', version=1)
```

2. 使用K-means聚类算法对标准正态分布的模拟数据集、鸢尾花数据集进行聚类分析。使用sklearn.datasets中的make_blobs函数生成标准正态分布的数据集。鸢尾花数据集被内置为一个示例数据集，可以直接加载使用。

```
from sklearn.datasets import load_iris
```

扩展思考

1. PCA与自编码器在降维中的对比案例

(1) 拓展背景：

主成分分析是线性降维的经典方法，而自编码器作为深度学习中的无监督模型，可通过非线性变换实现更复杂的数据压缩。对比两者在不同数据分布下的降维效果，能帮助理解线性与非线性降维的本质差异。

(2) 学习目标：

理解自编码器的编码 - 解码原理与 PCA 的线性变换差异。

对比两者在保留数据特征与抗噪声能力上的表现。

掌握深度学习模型在无监督降维中的应用范式。

2. DBSCAN 聚类算法与 K-means 的对比案例

(1) 拓展背景:

K-means 作为基于距离的聚类算法，适用于球形簇数据，但对噪声和非凸形状簇的处理能力有限。密度峰值聚类基于数据密度分布，能自动识别噪声点并处理任意形状的簇，是 K-means 的重要补充。

(2) 学习目标:

理解 DBSCAN 算法的密度可达与核心点概念。

对比两种算法在不同数据分布下的聚类效果。

掌握聚类算法参数调优与性能评估方法。